

Verteilte Systeme – Übungsblatt 1

1 Einleitung und Aufgabenstellung

Der Hauptbestandteil des ersten Übungsblatts ist eine verteilte Anwendung, in der n Prozesse einen logischen Ring bilden und ein Token (Streichholz) kreisen lassen. Der jeweilige Token-Besitzer zündet mit Wahrscheinlichkeit p einen Broadcast an alle Teilnehmer (Feuerwerksrakete) und reicht das Token anschließend weiter. Mit jedem Durchlauf halbiert ein Prozess seine Zündwahrscheinlichkeit ($p = p/2$). Die Anwendung terminiert, sobald in k aufeinanderfolgenden Runden keine Rakete mehr gezündet wurde.

Dasselbe Protokoll wurde in drei Umgebungen realisiert: pseudo-verteilt auf einem Rechner (Aufgabe 1), real verteilt über mehrere Computer (Aufgabe 2) und im Simulator sim4da (Aufgabe 3). Aufgabe 4 untersucht die Konsistenz der Anwendung. Den algorithmischen Kern bildet in allen Fällen der Token-Ring-Algorithmus nach Le Lann (1977): Da ausschließlich der Token-Besitzer zünden darf, ist der wechselseitige Ausschluss der Zündaktion bereits durch die Konstruktion gewährleistet.

Einsatz von KI-Werkzeugen. Bei der Bearbeitung wurde der KI-Assistent Claude Fable 5 sowie Opus 4.8 (Anthropic) als Hilfsmittel verwendet. Konkret unterstützte er beim Verständnis von Grundlagenwissen (UDP-Kommunikation), beim Entwurf und der Strukturierung des Quellcodes, beim Aufbau der Mess- und Auswertungsskripte sowie der Interpretation von Ergebnissen. Zusätzlich unterstützte der KI-Assistent bei der Diagnose der aufgetretenen Schwierigkeiten, insbesondere der schrittweisen Eingrenzung der Router-Client-Isolation in Aufgabe 2 und der Einrichtung der Java-Umgebung in Aufgabe 3. Die Ausführung aller Programme, die Durchführung der Experimente und die Erhebung der Messwerte erfolgten eigenständig auf der beschriebenen Hardware. Sämtliche dargestellten Zahlen stammen aus diesen eigenen Läufen. Die Ergebnisse wurden auf Plausibilität geprüft (etwa der Abgleich der gesendeten Raketen mit dem theoretischen $2n$ -Modell) und die Interpretation sowie die konzeptionellen Entscheidungen liegen in eigener Verantwortung.

2 Entwurf der Anwendung

Jeder Prozess durchläuft denselben Zyklus: Er wartet auf das Token seines Vorgängers, trifft im Token-Besitz (dem kritischen Abschnitt) seine Zündentscheidung und leitet das Token an den Nachfolger weiter. Prozess P_0 nimmt eine leichte Sonderrolle ein: Er erzeugt das initiale Token, zählt die Runden und löst nach k stillen Runden die Terminierung per STOP-Broadcast aus. Diese bewusste Asymmetrie vereinfacht Rundenzählung und Terminierungserkennung deutlich.

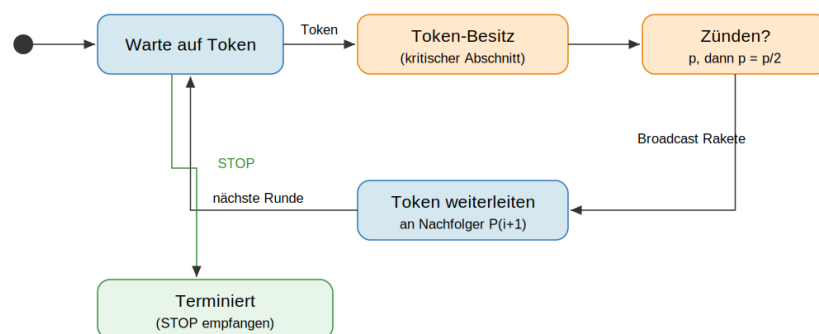


Abbildung 1: Zustandsdiagramm eines Ring-Prozesses (UML).

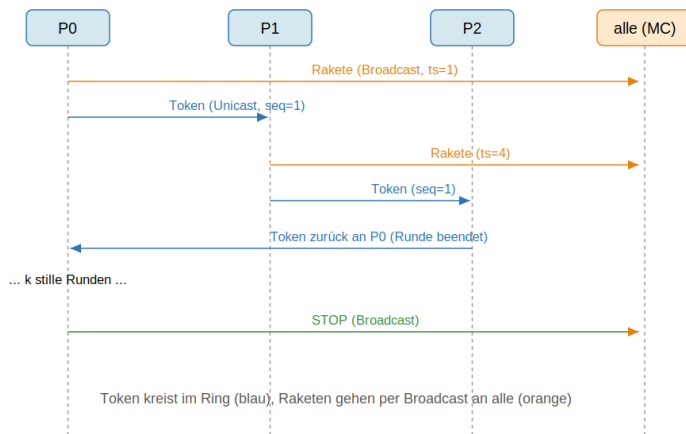


Abbildung 2: Sequenzdiagramm eines Token-Umlaufs mit Raketen-Broadcasts.

3 Aufgabe 1 – Pseudo-verteilt (localhost)

3.1 Implementierung

Die Implementierung erfolgte in Python. Der Ring kommuniziert über UDP-Unicast und die Ringtopologie ergibt sich vollständig aus einer Port-Arithmetik (Prozess i lauscht auf Port $50000 + i$, Nachfolger ist $50000 + (i+1) \bmod n$). Die Raketen werden auf UDP-Multicast abgebildet. Ein Skript automatisiert den Aufbau wachsender Ringe und aggregiert die Messwerte.

Behandlung der klassischen Token-Probleme: Da UDP keine Zustellung garantiert, bestätigt der Empfänger jede empfangene Token-Übergabe mit einer kurzen Quittungsnachricht (ACK, Acknowledgement) an den Sender. Erhält der Sender innerhalb einer festen Frist kein ACK, wertet er das Token als verloren und überträgt es erneut (Retransmit); eine pro Übergabe eindeutige Transfer-ID stellt dabei sicher, dass ein verspätet ankommendes Duplikat verworfen wird. Eine zunächst erwogene Timeout-Regeneration des Tokens wurde verworfen, da sie ein lediglich verzögertes Token verdoppeln könnte und damit die Token-Eindeutigkeit verletzt. Stattdessen wird das Token genau einmal erzeugt und Retransmit-Duplikate filtert eine pro Hop wachsende Transfer-ID. Im Fehlerfall wird damit bewusst die Terminierung geopfert, niemals jedoch die Sicherheit. Ein abgebrochener Lauf ist sichtbar und wiederholbar, eine verletzte Ausschluss-Garantie hingegen würde die Messung unbemerkt verfälschen.

3.2 Ergebnisse

Tabelle 1 zeigt die gemittelten Messwerte (drei Wiederholungen je n). Das maximale erfolgreiche n betrug 1024. Die Gesamtzahl der gesendeten Multicasts folgt sehr genau dem theoretischen Erwartungswert von $2n$ (vgl. Abbildung 4): Da jeder Prozess mit $p = 1, \frac{1}{2}, \frac{1}{4}, \dots$ zündet, beträgt die erwartete Raketenanzahl pro Prozess $\sum (\frac{1}{2})^i = 2$. Die Rundenzahl liegt nahe $\log_2(n) + k$.

n	Runden	Multicasts	t_min [ms]	t_mittel [ms]	t_max [ms]
2	4.3	6	0.07	0.27	0.8
4	7.3	11	0.16	0.54	2.0
8	6.7	17	0.34	1.27	4.5
16	8.0	35	0.65	3.47	13.0
32	10.7	63	1.33	8.84	44.7
64	12.0	136	2.76	39.18	215.0
128	11.0	255	8.27	211.74	1087.0
256	13.3	513	25.96	668.94	4496.1

512	11.7	1011	50.60	2718.57	16310.6
1024	15.0	2038	11.36	7445.46	62729.1

Tabelle 1: Messwerte Aufgabe 1 (localhost), Mittel aus je drei Läufen. Zustellrate durchgängig 100 %.

Bestimmung des maximalen n: Auffällig war, dass der erste beobachtete Abbruch bei $n = 2048$ kein echtes Systemlimit war, sondern ein Fehler des Messaufbaus: Ein zu knapp bemessener Idle-Timeout beendete früh gestartete Prozesse, bevor das Token überhaupt zu kreisen begann. Die Diagnose gelang über die Indizien (Abbruch nach exakt der Timeout-Dauer, somit kein Absturz und keine Retransmits). Nach Korrektur lief $n = 1024$ stabil. Das tatsächliche Limit setzt der Speicherbedarf und das Scheduling von über tausend Betriebssystemprozessen, nicht das Netzwerk und wird belegt durch durchgängig null Retransmits und 100 % Zustellrate.

4 Aufgabe 2 – Real verteilt

Für die verteilte Ausführung wurde die Topologie von der Port-Arithmetik auf eine gemeinsame Konfigurationsdatei umgestellt, die auf allen Rechnern identisch vorliegt und die Ringreihenfolge als Liste von IP-Adressen und Ports enthält. Die Broadcast-Schicht ist umschaltbar zwischen UDP-Multicast und dem $n-1$ Unicasts.

4.1 Schwierigkeiten mit realer Infrastruktur

Dieser Teil erforderte den mit Abstand höchsten Aufwand, jedoch nicht algorithmischer, sondern infrastruktureller Natur. Zur Verfügung standen zwei Laptops (nur WLAN) und ein Desktop-Rechner (nur LAN). Eine systematische Fehlersuche schloss nacheinander mehrere Ursachen aus: lokale Firewalls (auch bei deaktivierter Firewall kam kein UDP an), Subnetz-Fehlkonfiguration (identische Subnetzmaske), ein virtueller VirtualBox-Adapter sowie ein vermeintlicher Code-Fehler. Ein Loopback-Test (Sender und Empfänger auf demselben Rechner) belegte die Korrektheit des Codes; ein UDP-Paket kam dort sofort an, scheiterte aber zwischen je zwei Rechnern.

Als Ursache wurde die **Client-Isolation** des Heimrouters identifiziert: Dieser leitet ICMP (Ping) weiter, blockiert aber direkte UDP-Kommunikation zwischen seinen Clients. Da kein Zugang zur Router-Konfiguration bestand, wurde der Router umgangen, indem ein Laptop per Windows-Hotspot ein eigenes, isolationsfreies WLAN aufspannte. Der Desktop-Rechner ließ sich mangels WLAN-Fähigkeit nicht einbinden; die Ausführung erfolgte daher nur mit zwei Laptops.

4.2 Ergebnisse und Vergleich der Broadcast-Verfahren

Verfahren	Zustellrate	Retransmits	Runden	t min/mittel/max [ms]
Unicast	100 %	0	10	3,87 / 7,46 / 23,24
Multicast	50 %	2	7	3,66 / 103,29 / 695,10

Tabelle 2: Verteilter Ring mit $n = 2$ über WLAN-Hotspot

Über WLAN erreichte Unicast eine Zustellrate von 100 % und Multicast nur 50 %. Der Grund liegt in der Funkschicht: Unicast-Calls werden per Link-Layer-ACK bestätigt und bei Verlust wiederholt, Multicast-Frames hingegen ungesichert gesendet. Der Unicast-Fallback ($n-1$ Sendungen pro Rakete) ist über WLAN somit zuverlässiger und schneller. Gegenüber localhost (Aufgabe 1, $n = 2$: ca. 0,28 ms) stieg die mittlere Rundenzeit über WLAN auf 7,46 ms, etwa das 25-Fache, bei zugleich deutlich höherer Streuung.

5 Aufgabe 3 – Simulation mit sim4da

Die Simulation nutzt den Simulator sim4da (Java), in dem alle Knoten als Threads innerhalb einer JVM laufen und Nachrichten zuverlässig in-memory zugestellt werden. Das Protokoll wurde strukturgleich zur Python-Fassung gehalten. Send() bildet den Token-Ring und broadcast() die Raketen. Die Terminierung erfolgt wie

vom Framework gefordert, ausschließlich über Nachrichten. Auffällig ist, dass der Simulator-Code deutlich kürzer ausfällt. Die gesamte ACK/Retransmit-Logik entfällt, da der Kanal zuverlässig ist.

Maximales n : Die Simulation erreichte $n = 8192$, also das Achtfache der realen Prozessgrenze aus Aufgabe 1. Da jeder Knoten ein Thread ist, konnte durch verkleinerte Thread-Stacks (-Xss) und erhöhten Heap (-Xmx) diese Größe erreicht werden. Begrenzend war hier nicht der Speicher, sondern die Laufzeit: Die Zahl der Zustellungen wächst quadratisch mit n (bei $n = 8192$ über 134 Millionen Raketen-Empfänge), wodurch ein einzelner Lauf bis zu 44 Minuten benötigte.

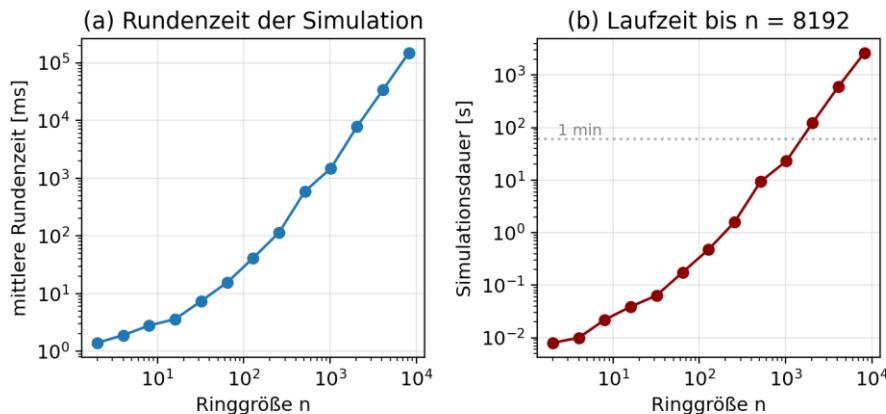


Abbildung 3: Rundenzeit (a) und Gesamtlaufzeit (b) der Simulation bis $n = 8192$.

5.1 Vergleich der drei Realisierungen

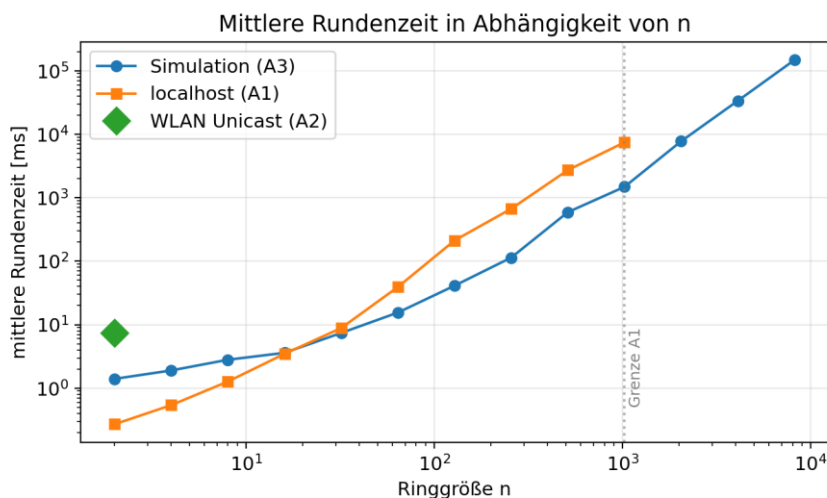


Abbildung 4: Mittlere Rundenzeit aller drei Umgebungen (doppelt-logarithmisch).

Abbildung 4 fasst den zentralen Befund zusammen. Es zeigt sich ein **Crossover**: Bei kleinem n ist die Simulation langsamer als localhost (bei $n = 2$: 1,4 ms gegenüber 0,28 ms), da der Overhead der Thread-Synchronisation dominiert. Ab etwa $n = 32$ kehrt sich das Verhältnis um; bei $n = 1024$ ist die Simulation rund 4,5-mal schneller (1,48 s gegenüber 7,45 s), weil echte Betriebssystemprozesse mit echten Sockets stärker unter Kontextwechseln leiden als In-Memory-Threads. Das reale WLAN (Aufgabe 2) ist bereits bei $n = 2$ um eine Größenordnung langsamer als beide lokalen Varianten.

Keine Umgebung ist demnach absolut überlegen: Die Simulation skaliert am weitesten, misst aber keine reale Latenz. Localhost ist bei kleinem n am schnellsten, stößt aber früher an Ressourcengrenzen. Das reale Netz ist am langsamsten und unzuverlässigsten, besitzt aber als einzige echte verteilte Semantik. Auch der Aufwand unterscheidet sich: Die Simulation erforderte den geringsten Implementierungsaufwand (keine

Fehlertoleranz), aber hohen Experimentalaufwand bei großem n . Die reale Variante erforderte den höchsten Gesamtaufwand (Sockets, Zuverlässigkeit, Netzwerkd Diagnose).

6 Aufgabe 4 – Konsistenz

Die zentrale Frage lautet, ob die beteiligten Prozesse eine inkonsistente Sicht auf den Programmablauf haben können. Dafür wurden drei Konsistenzkriterien definiert und die Simulation aus Aufgabe 3 um Mechanismen erweitert, die Verletzungen sowohl erkennen als auch vermeiden.

K1 – Token-Eindeutigkeit (Safety). Zu jedem Zeitpunkt existiert höchstens ein Token. Das Token trägt eine pro Runde wachsende Sequenznummer. Ein Token mit kleinerer Nummer wird als veraltetes Duplikat erkannt und verworfen.

K2 – Einheitliche Sicht auf Raketen (Agreement). Alle Prozesse sollen dieselbe Menge gezündeter Raketen sehen. Dieses Kriterium wurde in Aufgabe 2 real verletzt gemessen: über WLAN-Multicast lag die Zustellrate bei nur 50 %.

K3 – Kausale Ordnung (erweiterte Lamport-Zeit). Jede Nachricht trägt einen Lamport-Zeitstempel. Die Uhr wird bei Empfang auf $\max(\text{eigene, fremde}) + 1$ gesetzt. Im Erkennungs-Modus wird eine Verletzung gemeldet, sobald eine Rakete mit kleinerem Zeitstempel nach einer bereits ausgelieferten eintrifft. Im Vermeidungs-Modus werden Raketen in einer nach Zeitstempel/ Sender-ID sortierten Warteschlange gepuffert und erst dann ausgeliefert, wenn von jedem anderen Sender bereits eine spätere Nachricht vorliegt. Dies entspricht der Auslieferungsregel von Lamports verteilter Warteschlange.

Um Verletzungen überhaupt sichtbar zu machen, wurde der ansonsten zuverlässige Simulator künstlich gestört: durch Nachrichtenverlust (verletzt K2) und durch Umordnung empfangener Raketen (verletzt K3). Tabelle 3 zeigt die Ergebnisse für $n = 8$.

Verlust	Umordnung	Modus	Zustellrate	K1	K3
0 %	0 %	detect	100 %	0	0
0 %	40 %	detect	100 %	0	32
0 %	40 %	avoid	100 %	0	0
30 %	0 %	detect	66,7 %	0	0

Tabelle 3: Konsistenz-Experimente ($n = 8$). K1/K3 = Anzahl gemeldeter Verletzungen.

Die Ergebnisse belegen vier Aussagen. Erstens meldet der ungestörte Lauf keine Verletzung, der Simulator selbst ist also konsistent. Zweitens senkt Nachrichtenverlust die Zustellrate auf 66,7 % und verletzt damit K2, ohne die Ordnung (K3) zu berühren. Drittens erzeugt Umordnung 32 K3-Verletzungen, die der Detektor zuverlässig meldet. Viertens reduziert der Vermeidungs-Mechanismus dieselbe Umordnung auf null Verletzungen. Verschiedene Störungen verletzen also verschiedene Kriterien.

Schlussfolgerung: Konsistenz ist keine Eigenschaft des Algorithmus, sondern des Zusammenspiels von Algorithmus und Kanal. Derselbe Le-Lann-Ring ist im zuverlässigen, geordneten Simulator vollständig konsistent, über reales UDP/WLAN hingegen verletzlich. Die eingebauten Mechanismen (Sequenznummer für K1, Lamport-Uhren mit Hold-back-Warteschlange für K3) stellen die Garantien wieder her, die der Kanal nicht bietet, zum Preis von zusätzlichem Code und im Vermeidungs-Modus zusätzlicher Wartezeit.

7 Fazit

Die Bearbeitung zeigt, dass ein einfacher Token-Ring-Algorithmus sich in drei Umgebungen fundamental unterschiedlich verhält. Die Simulation skaliert am weitesten ($n = 8192$) bei perfekter Zuverlässigkeit, jedoch ohne reale Latenz. Die lokale Variante ist bei kleinem n am schnellsten während die reale verteilte

Ausführung am aufwändigsten und unzuverlässigsten ist, dafür aber alleine realitätsnah. Der größte praktische Aufwand entstand nicht im Algorithmus, sondern in der realen Infrastruktur (Router-Client-Isolation, Hardware-Grenzen). Die Konsistenzbetrachtung verbindet alle Teile: Sie macht deutlich, dass die in der Simulation vorhandenen Garantien in der Realität aktiv durch Mechanismen wie Lamport-Uhren und zuverlässige Übergabe hergestellt werden müssen.